1. Échelle de croissance

$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n!$

Repères : $2^{10} \approx 10^3$, $2^{20} \approx 10^6$. Pour $n = 10^6$: $\log_2 n \approx 20$, $\sqrt{n} = 1000$, $n \log n \approx 2 \cdot 10^7$, $n^2 = 10^{12}$.

Complexité = **pire cas** par défaut. Big-O ignore les constantes : $100n = O(n)$, $n/2$ tours $= O(n)$.

2. Lire une boucle

Forme	Tours
`i ← i + 1` jusqu'à n	n
`i ← i × 2` ou `i ← i / 2`	$\log_2 n$
`while i × i ≤ n, i ← i + 1`	$\sqrt{n}$
`for j ← 1 to 100`	constante, O(1)

Boucles indépendantes : multiplier. Borne interne qui dépend de i : additionner.

Somme	Total
$1 + 2 + \ldots + n$	$n(n+1)/2 = O(n^2)$
$1^2 + 2^2 + \ldots + n^2$	$\approx n^3/3 = O(n^3)$
$1 + 2 + 4 + \ldots + n$	$\approx 2n = O(n)$
$n + n/2 + n/4 + \ldots + 1$	$\approx 2n = O(n)$
$n + n/2 + n/3 + \ldots + 1$	$\approx n \ln n = O(n \log n)$
$\log 1 + \log 2 + \ldots + \log n$	$O(n \log n)$

Piège : une boucle en log n dont le travail interne double n'est PAS n log n. Écrire la somme.

Piège : « valeur finale de sum » ≠ complexité. Compter les passages dans le if.

3. Récursion : mot à mot

Sous-problèmes	Combinaison	Résultat
1 de taille n/2	O(1)	O(log n), dichotomie
1 de taille n/2	O(n)	O(n)
2 de taille n/2	O(1)	O(n)
2 de taille n/2	O(n)	O(n log n), fusion
1 de taille n−1	O(1)	O(n)
1 de taille n−1	O(n)	$O(n^2)$, rapide au pire
2 de taille n−1	O(1)	$O(2^n)$, Fibonacci naïf

Méthode : dessiner l'arbre des appels, coût par niveau × nombre de niveaux.

Espace d'une récursion = profondeur max, pas le nombre d'appels.

4. Fibonacci et exponentiation

	Temps	Espace
Naïf récursif	$O(\phi^n)$, majoré par $O(2^n)$	O(n)
Mémoïsé (DP)	O(n)	O(n)
Itératif, 2 variables	O(n)	O(1)

Mémoïsation : tester `memo[n]`, puis **écrire** `memo[n]` avant de retourner.
Appels naïfs : $c(n) = 1 + c(n-1) + c(n-2)$, $c(5) = 15$.

Exponentiation rapide en O(log n) : cas de base $\mathbf{x^0 = 1}$; half ← pow(x, n//2) ; pair : half × half ; impair : half × half × x.

Crible d'Ératosthène $O(n \log \log n)$. Test jusqu'à $\sqrt{i}$: $O(n\sqrt{n})$ au total.

5. P, NP

- **P** : résoluble en temps polynomial. **NP** : solution **vérifiable** en temps polynomial (NP ≠ « non polynomial »).
- **NP-complet** : dans NP, et tout NP s'y réduit. Exemple : Sudoku, TSP en version décision.
- **NP-difficile** : au moins aussi dur, pas forcément dans NP. Exemple : TSP en version optimisation, échecs généralisés.
- Un NP-complet résolu en temps polynomial ⇒ P = NP. Question ouverte.

6. Structures : coûts

Structure	Tête / début	Queue / fin	Milieu	Recherche
Tableau	insert O(n)	O(1) si place	accès O(1), insert O(n)	O(n), trié O(log n)
Liste simple	O(1)	O(n)	O(n)	O(n)
Liste double	O(1)	O(1)	O(n)	O(n)
Liste circulaire	O(1)	O(n)	O(n)	O(n)
Pile (LIFO)	push, pop, peek O(1) au sommet seulement			O(n)
File (FIFO)	enqueue en queue, dequeue en tête, O(1)			O(n)
Hachage	insert, recherche, suppression : O(1) en moyenne, O(n) au pire, sans ordre			

Liste simple avec `tail` : suppression du dernier toujours O(n), il faut l'avant-dernier.

Adresse : `a[i] = base + i × taille`. Attention à l'hexa : 20 = 0x14.

Collisions : chaînage, adressage ouvert (case libre suivante), re-hachage.

Struct : `p.age` ; pointeur `pp->age` = `(*pp).age`.

7. Du scénario à la structure

Mots de l'énoncé	Choix
dernier ajouté, annuler, appels, parenthèses	Pile
ordre d'arrivée, impression, BFS	File
clé → valeur, recherche par nom, ordre égal	Table de hachage
accès direct par indice, taille fixe	Tableau
ajouts et suppressions aux deux bouts	Liste double
priorité, le plus urgent d'abord	File de priorité (tas)
hiérarchie, dossiers, expression	Arbre
routes à sens unique avec distance	Graphe orienté pondéré
amis, co-apparition, sans sens ni coût	Graphe non orienté

8. Pile d'appels

Le code **avant** l'appel récursif s'exécute à la descente, le code **après** à la remontée, en ordre inverse.

```
f(n) : print n ; f(n-1)  →  3 2 1 END
f(n) : f(n-1) ; print n  →  END 1 2 3
```

Leçons du sujet 2025

- Le sujet annonce 20 questions, il y en a 19 : 6 complexité, 3 structures, 1 tri, 9 graphes et arbres.
- Boucle `j ← i` puis `j ← j × 2` : le corrigé donne O(n log n), mais la somme exacte vaut ≈ 2n, soit O(n).
- Scénario « fin + milieu + début » : le corrigé choisit la liste double, alors que le milieu y coûte O(n). Le prof privilégie ici les ajouts et suppressions aux extrémités.
- Questions de code : le bug est souvent un cas de base.
- Arbres et AVL : regarder chaque nœud, pas seulement la racine.

9. Recherche

Méthode	Données	Temps	Espace
Linéaire	quelconques	O(n)	O(1)
Par sauts, m = √n	triées	O(√n)	O(1)
Binaire	triées	O(log n)	O(1) itér., O(log n) réc.

Sauts : coût n/m + m, minimal pour m = √n, soit ≈ 2√n comparaisons.

```
low ← 0 ; high ← n − 1
while low ≤ high do          ← ≤ et non <
    mid ← (low + high) // 2
    A[mid] < x : low ← mid + 1
    A[mid] > x : high ← mid − 1
```

Plusieurs recherches sur données non triées : trier une fois, puis dichotomie.

10. Tris

Tri	Meilleur	Moyen	Pire	Espace	Stable
Sélection	n^2	n^2	n^2	O(1)	non
Insertion	n	n^2	n^2	O(1)	oui
Bulles	n (arrêt)	n^2	n^2	O(1)	oui
Fusion	n log n	n log n	n log n	O(n)	oui
Rapide	n log n	n log n	n^2	O(log n)	non

- **Sélection** : toujours n(n−1)/2 comparaisons. Après k passes, les k plus petits sont à gauche.
- **Insertion** : le préfixe est trié. Idéal sur tableau presque trié. Pire cas : trié à l'envers.
- **Bulles** : compare les voisins. Après k passes, les k plus grands sont à droite.
- **Fusion** : deux tableaux triés de tailles a et b, au plus a + b − 1 comparaisons.
- **Rapide** : pivot = dernier élément ; indice final = nombre d'éléments plus petits. Tableau déjà trié ⇒ $O(n^2)$.

Plus petite différence, doublons : **trier puis comparer les voisins**, O(n log n).

11. Arbres

- n sommets, **n − 1 arêtes**, connexe, sans cycle, un chemin unique entre deux sommets.
- Retirer une arête : 2 composantes. Ajouter une arête : exactement 1 cycle. Sommet de degré 0 (n > 1) : pas un arbre.
- Racine au **niveau 0**. Hauteur = niveau maximal. Feuille = sans enfant.
- Niveau k : au plus 2^k nœuds. Parfait de hauteur h : **2^(h+1) − 1** nœuds.
- **Complet** : tous les niveaux pleins sauf le dernier, rempli à gauche. **Parfait** : tout est plein. **Dégénéré** : une chaîne.
- **BST** : gauche < nœud < droite, récursivement. Insertion depuis la racine, un nœud placé ne bouge plus. O(log n) équilibré, O(n) dégénéré (valeurs triées).
- Expression : l'opération faite en dernier est à la racine.

12. AVL : méthode sans erreur

h(vide) = 0, feuille = 1, h = 1 + max(hg, hd). FE = hg − hd ∈ {−1, 0, +1} pour CHAQUE nœud.

« Est-ce un AVL ? » Depuis les feuilles, une ligne par nœud : `nœud : hg, hd → FE, h`. Un seul ±2 ⇒ non. Jamais à l'œil.

Insertion, à chaque valeur :

1. Placer comme dans un BST, en comparant depuis la racine.
2. Remonter vers la racine en recalculant h et FE.
3. Au **premier** FE = ±2 (le plus **bas**) : c'est z, on s'arrête. Ne pas tourner la racine si z est plus bas.
4. Faire les **2 pas** de z vers la valeur insérée :

2 pas	Forme	Action
G, G	ligne	rotation droite sur z
D, D	ligne	rotation gauche sur z
G, D	**zigzag**	rot. gauche sur l'enfant, puis droite sur z
D, G	**zigzag**	rot. droite sur l'enfant, puis gauche sur z

Rotation gauche sur z (y = enfant droit) : y prend la place de z, z devient l'enfant gauche de y, l'ancien enfant gauche de y passe à droite de z. Rotation droite : symétrique.

Raccourci : parmi z, son enfant et son petit-enfant, la **médiane monte**, les deux autres vont dessous, les sous-arbres restants se raccrochent dans l'ordre croissant. Puis **recalculer tous les FE** : la racine se répare seule.

Pièges : rotation simple sur un zigzag (il reste un ±2) ; tourner la racine au lieu du plus bas ; oublier de vérifier après la dernière insertion.

Ex. : 10, 20, 30 (D, D : 20 monte), puis 25, 28 : 30 a FE = +2, pas G puis D ⇒ zigzag, 28 monte ⇒ 20 (10, 28 (25, 30)). Insérer 1 à 7 ⇒ arbre parfait de racine 4.

13. Graphes

	Matrice	Liste d'adjacence
Mémoire	$O(V^2)$	O(V + E)
Test d'arête	O(1)	plus lent
Pour	graphe dense	graphe peu dense

- Non orienté : matrice **symétrique**, somme des degrés = **2E**.
- Orienté : case (i, j) = arête i → j. Pondéré : la case vaut le poids.
- Vérifier orienté ? pondéré ? cyclique ? connexe ?

	BFS (largeur)	DFS (profondeur)
Structure	file	pile ou récursion
Temps	O(V + E)	O(V + E)
Usage	plus court chemin non pondéré, niveaux	cycles, tri topologique, labyrinthe

BFS à la main : défiler u, écrire u, enfiler ses voisins non visités (alphabétique), en les marquant dès l'enfilement.

DFS à la main : écrire u, aller au **premier** voisin non visité ; sinon remonter.

14. Arbres et graphes : réflexes

- **Orienté** : BFS et DFS ne suivent que les flèches sortantes. Un sommet qui pointe vers le départ n'est jamais visité.
- Somme des degrés = 2E, donc **toujours paire** : degrés 3, 3, 3, 2, 2 ⇒ graphe impossible.
- Connexe + n − 1 arêtes ⇒ arbre. Sans cycle + n − 1 arêtes ⇒ arbre.
- Recherche dans un BST : nœuds visités = niveau de la cible + 1. Minimum : tout à gauche ; maximum : tout à droite.

15. En salle

- 3 minutes par question. Pas de point négatif : **tout cocher**.
- Passer les AVL et parcours longs, y revenir, marquer les sommets au brouillon.
- Sur une correction de code : tester n = 0, n = 1, 2 éléments.
- Options de type « presque juste » : relire les bornes des boucles.